


**Laboratorium**
**Robotika dan Sistem Cerdas**
**Departemen Teknik Komputer**
***Institut Teknologi Sepuluh Nopember***

# Laporan Akhir
# Praktikum Sistem Mikroprosesor dan Sistem Tertanam

## PENGANTAR MIKROKONTROLER

Najib Fathir Rizqi - 5024241088

2026

# 1 Langkah-Langkah Percobaan

## 1.1 Percobaan 1: Pengenalan STM32 Blackpill (Blink)

1. Membuka kembali project Modul 1 untuk board STM32 Blackpill, kemudian menyesuaikan konfigurasi pada berkas `platformio.ini` seperti berikut.

```
[env:blackpill_f401cc]
platform = ststm32
board = blackpill_f401cc
framework = arduino
upload_protocol = stlink
debug_tool = stlink
```

2. Menghubungkan pin pada board STM32 Blackpill dengan ST-Link menggunakan kabel jumper. Setelah itu, ST-Link disambungkan ke komputer melalui USB dengan memperhatikan kecocokan setiap pin.

3. Memasukkan program Blink yang telah disiapkan ke dalam berkas `src/main.cpp`. Selanjutnya, menjalankan proses *Build* dan *Upload* melalui PlatformIO.

4. Mengamati LED bawaan pada pin PC13. LED akan menyala selama 500 ms, kemudian padam selama 500 ms secara berulang.

## 1.2 Percobaan 2: Pengenalan ESP32 dengan Framework ESP-IDF (Blink)

1. Menggunakan project Modul 1 untuk ESP32, lalu memperbarui konfigurasi `platformio.ini` menjadi seperti berikut.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = espidf
```

2. Menyambungkan board ESP32 ke komputer menggunakan kabel USB dan memeriksa apakah perangkat telah terdeteksi pada port yang tersedia.

3. Menghapus berkas `src/main.cpp`, kemudian membuat berkas baru bernama `src/main.c`. Kode Blink yang disediakan dimasukkan ke dalam berkas tersebut, lalu dijalankan proses *Build* dan *Upload*.

4. Memeriksa LED bawaan pada GPIO2. LED diamati menyala selama setengah detik dan mati selama setengah detik, kemudian mengulangi pola tersebut.

## 1.3 Percobaan 3: Pull-up dan Pull-down Eksternal dan Internal pada ESP32

### 1.3.1 Eksperimen 1: Pull-up dan Pull-down Eksternal

1. Mengubah kembali konfigurasi project ESP32 pada `platformio.ini` agar menggunakan framework Arduino. Berkas `src/main.c` kemudian diganti menjadi `src/main.cpp`.

```
[env:esp32dev]
platform = espressif32
board = esp32dev
framework = arduino
```

2. Merangkai dua pushbutton pada GPIO 32 dan GPIO 33 beserta resistor eksternal berdasarkan Skema Bagian A pada modul.
3. Memasukkan program yang telah disediakan ke dalam `src/main.cpp`, kemudian melakukan proses kompilasi dan mengunggah program ke ESP32.
4. Menguji kedua pushbutton dengan menekannya secara bergantian maupun bersamaan. Setiap perubahan kondisi diamati melalui Serial Monitor dan hasilnya dicatat.

#### 1.3.2 Eksperimen 2: Pull-up dan Pull-down Internal

1. Melepaskan resistor eksternal dari rangkaian, kemudian menyusun kembali koneksi tombol mengikuti Skema Bagian B.
2. Memperbarui isi berkas `src/main.cpp` menggunakan program untuk konfigurasi pull-up dan pull-down internal. Setelah itu, melakukan *Build* serta *Upload*.
3. Melakukan pengujian dengan menekan masing-masing tombol secara terpisah dan bersamaan. Kondisi logika yang muncul pada Serial Monitor selanjutnya diamati dan didokumentasikan.

### 1.4 Percobaan 4: Debouncing pada Input Tombol GPIO (ESP32 dengan Arduino)

1. Mempertahankan konfigurasi `platformio.ini` dari percobaan sebelumnya, kemudian merangkai komponen sesuai diagram rangkaian untuk pengujian debouncing.
2. Menuliskan kode percobaan pada berkas `src/main.cpp`, lalu melakukan kompilasi dan mengunggahnya ke board ESP32.
3. Menekan tombol satu kali dan membandingkan hasil penghitung `counterNoDebounce` dengan `counterWithDebo` yang ditampilkan melalui Serial Monitor.
4. Mengatur konstanta `DEBOUNCE_DELAY` menjadi 10 ms. Pengujian tombol dilakukan kembali dan hasil kedua penghitung dicatat.
5. Mengubah nilai `DEBOUNCE_DELAY` menjadi 200 ms, kemudian mengulangi pengujian untuk melihat pengaruh perubahan waktu tunda terhadap pembacaan tombol.

### 1.5 Percobaan 5: Pembacaan ADC Menggunakan LDR pada ESP32

1. Menggunakan kembali pengaturan project dari percobaan sebelumnya. Rangkaian LDR dan resistor disusun sesuai skema percobaan, dengan titik keluaran pembagi tegangan dihubungkan menuju GPIO34.
2. Memasukkan kode pembacaan ADC ke dalam `src/main.cpp`, kemudian menjalankan proses *Build* dan *Upload* pada PlatformIO.

3. Memantau nilai ADC yang dikirimkan ke Serial Monitor. Pengamatan dilakukan dengan memberikan kondisi cahaya yang berbeda, yaitu menutupi permukaan LDR menggunakan tangan dan mengarahkannya ke sumber cahaya.

## 2 Analisis Hasil Percobaan

Kegiatan praktikum dimulai dengan mencoba fungsi dasar dua jenis mikrokontroler, yaitu STM32 Blackpill dan ESP32, melalui program Blink. Dalam percobaan pertama, board STM32 Blackpill menggunakan framework Arduino atau STM32duino. Pengiriman program ke board dilakukan melalui ST-Link dengan protokol komunikasi SWD. Program disusun menggunakan fungsi `setup()` untuk menetapkan PC13 sebagai keluaran digital melalui `pinMode()`, sedangkan fungsi `loop()` menjalankan instruksi secara terus-menerus. Perintah `digitalWrite()` digunakan untuk mengubah kondisi LED dan `delay()` mengatur jeda waktunya. Hasil pengujian memperlihatkan LED berkedip dengan durasi 500 ms untuk setiap kondisi. LED pada PC13 menggunakan karakteristik aktif-LOW, sehingga kondisi LOW justru membuat LED menyala.

Pada percobaan kedua, program Blink diterapkan pada ESP32 dengan framework ESP-IDF. Struktur programnya berbeda dari Arduino karena titik awal aplikasi menggunakan fungsi `app_main()`, sedangkan pengulangan instruksi perlu dibuat secara eksplisit menggunakan `while(1)`. Pengaturan GPIO juga dilakukan melalui fungsi khusus, antara lain `gpio_reset_pin()`, `gpio_set_direction()`, dan `gpio_set_level()`. Selain itu, header yang berkaitan dengan driver GPIO dan FreeRTOS perlu disertakan. Penundaan program memanfaatkan `vTaskDelay(pdMS_TO_TICKS(500))`, yang memberikan kesempatan kepada task lain untuk berjalan selama waktu tunggu. Berbeda dengan itu, `delay()` pada Arduino menghentikan sementara alur eksekusi program. Implementasi ESP-IDF ditulis dalam berkas `main.c` dan proses kompilasi awal membutuhkan waktu kurang lebih 390 detik karena komponen framework ikut dibangun. LED ESP32 pada GPIO2 memiliki sifat aktif-HIGH, sehingga berbeda dengan LED STM32 pada percobaan pertama. Perbandingan ini memperlihatkan bahwa Arduino menawarkan penulisan program yang lebih sederhana, sedangkan ESP-IDF menyediakan pengaturan perangkat keras dan sistem operasi waktu nyata secara lebih terperinci.

Pengujian berikutnya berfokus pada GPIO sebagai masukan digital dengan memanfaatkan pushbutton. Dua metode penentuan kondisi logika diuji, yakni penggunaan resistor pull-up dan pull-down eksternal serta pemanfaatan resistor internal ESP32. Pada konfigurasi pull-up di GPIO32, kondisi tombol yang dilepas menghasilkan HIGH, sedangkan ketika ditekan berubah menjadi LOW. Sebaliknya, konfigurasi pull-down pada GPIO33 menghasilkan LOW saat tombol tidak ditekan dan HIGH ketika tombol ditekan. Pengamatan pada rangkaian eksternal maupun internal memperlihatkan pola logika yang sejalan. Dengan demikian, resistor internal yang diaktifkan melalui `INPUT_PULLUP` dan `INPUT_PULLDOWN` dapat digunakan sebagai pengganti resistor eksternal 10 k$\Omega$. Namun, nilai resistansi internal ESP32 yang berkisar 45 k$\Omega$ membuat gaya tariknya lebih lemah dibandingkan resistor eksternal.

Ketika konfigurasi internal diuji, Serial Monitor sempat menampilkan karakter yang tidak terbaca. Kondisi tersebut muncul karena kecepatan komunikasi serial pada program dan monitor tidak sama. Program menggunakan `Serial.begin(115200)`, sementara pengaturan monitor masih berada pada nilai bawaan 9600 bps karena parameter `monitor_speed` belum ditambahkan ke `platformio.ini`. Komunikasi UART tidak menggunakan clock bersama, sehingga kedua sisi harus memiliki baud rate yang cocok agar data diterima pada waktu yang benar. Ketidaksesuaian kecepatan dapat menye-

babkan bit dibaca pada posisi yang keliru dan menghasilkan karakter yang tidak sesuai. Masalah dapat diatasi dengan menyamakan baud rate program menjadi 9600 bps. Pada pengujian selanjutnya, parameter `monitor_speed = 115200` ditambahkan sehingga program dapat kembali menggunakan `Serial.begin(115200)`. Hal ini menunjukkan pentingnya menyamakan konfigurasi kecepatan komunikasi antara perangkat dan Serial Monitor.

Penggunaan resistor pull-up atau pull-down dapat menjaga kondisi logika pin agar tidak mengambang, tetapi belum tentu menjamin setiap penekanan tombol hanya dihitung satu kali. Oleh sebab itu, percobaan keempat membandingkan pembacaan tombol tanpa debouncing dan dengan debouncing pada beberapa nilai waktu tunda. Saat `DEBOUNCE_DELAY` bernilai 50 ms, penghitung tanpa debounce menunjukkan 11 kejadian, sedangkan penghitung dengan debounce mencatat 9 kejadian. Dua kejadian tambahan dapat dianggap sebagai akibat pantulan kontak tombol yang berhasil diabaikan oleh proses penyaringan. Ketika waktu tunda diturunkan menjadi 10 ms, kedua penghitung sama-sama mencapai 16. Hasil tersebut menunjukkan bahwa waktu penyaringan terlalu singkat untuk menghilangkan seluruh pantulan sinyal. Sebaliknya, ketika waktu tunda dinaikkan menjadi 200 ms, penghitung tanpa debounce mencapai 14, sementara penghitung dengan debounce tetap menunjukkan 0. Penekanan tombol yang dilakukan berulang dengan cepat tidak mampu mempertahankan sinyal dalam kondisi stabil selama 200 ms. Dengan demikian, nilai debounce yang terlalu besar dapat menyebabkan respons tombol terlambat atau bahkan tidak terdeteksi. Berdasarkan hasil pengamatan, 50 ms menjadi nilai kompromi yang paling sesuai pada pengujian ini.

Pada percobaan terakhir, GPIO digunakan sebagai masukan analog melalui ADC dengan bantuan komponen LDR. LDR dipasang bersama resistor 10 k$\Omega$ sehingga membentuk rangkaian pembagi tegangan. Tegangan keluaran pada titik tengah dapat dinyatakan sebagai

$$V_o = 3{,}3 \times \frac{10\,\text{k}\Omega}{R_{\text{LDR}} + 10\,\text{k}\Omega}.$$

Nilai resistansi LDR berubah mengikuti intensitas cahaya yang diterima. Akibatnya, tegangan keluaran rangkaian juga mengalami perubahan secara kontinu. Ketika permukaan LDR ditutup, pembacaan ADC berada pada kisaran 357–583 atau sekitar 0,29–0,47 V, yang menunjukkan resistansi LDR relatif tinggi, kurang lebih 100 k$\Omega$. Saat LDR memperoleh cahaya, nilai ADC meningkat menjadi sekitar 1867–3049 atau 1,50–2,46 V, seiring menurunnya resistansi hingga kurang lebih 3,4 k$\Omega$. Berbeda dari digital input yang hanya menghasilkan HIGH atau LOW berdasarkan ambang tegangan, ADC 12-bit ESP32 mengubah tegangan masukan menjadi nilai digital pada rentang 0–4095. Rentang tersebut menyediakan 4096 tingkat pembacaan dengan resolusi ideal sekitar 0,8 mV untuk rentang 3,3 V.

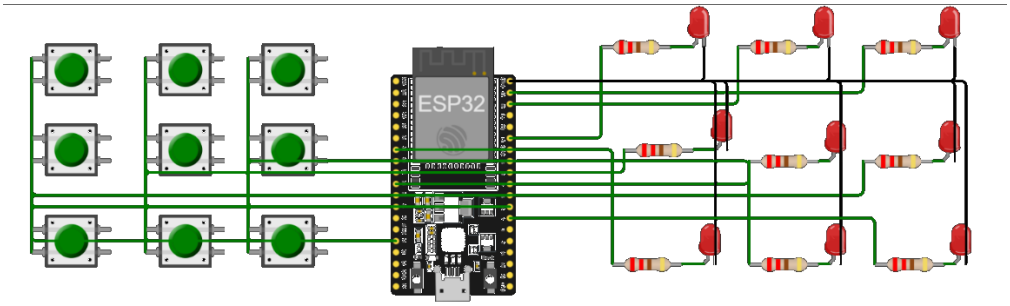
Secara umum, rangkaian kegiatan pada Modul 1 memperkenalkan tiga pemanfaatan GPIO, yaitu sebagai keluaran digital melalui program Blink, masukan digital melalui pushbutton beserta pengujian pull-up, pull-down, dan debouncing, serta masukan analog melalui pembacaan LDR menggunakan ADC. Hasil praktikum juga memperlihatkan bahwa pengoperasian mikrokontroler dipengaruhi oleh sejumlah pengaturan pendukung. Pemilihan framework, kesesuaian baud rate, serta penentuan waktu debounce merupakan beberapa hal yang perlu diperhatikan agar sistem dapat bekerja sesuai rancangan.

# 3 Hasil Tugas Modul

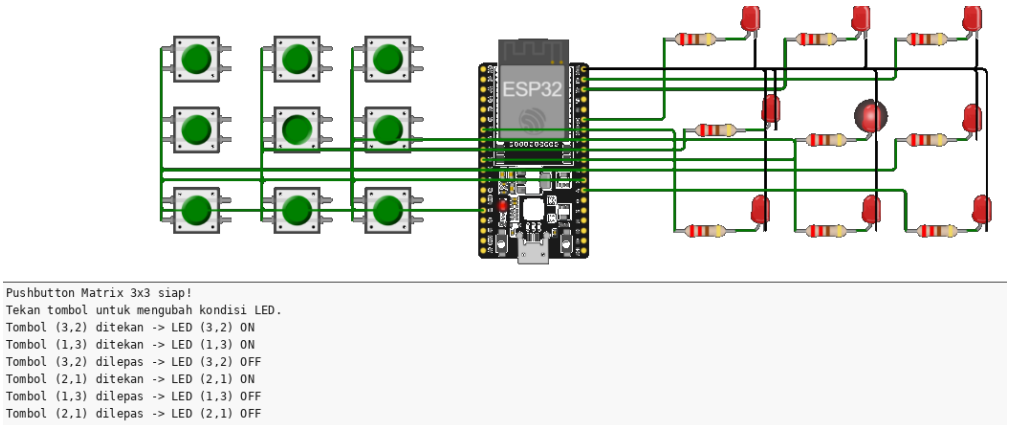
## 3.1 Hasil Simulasi dan Source Code Matrix Pushbutton 3×3 untuk Mengendalikan Matriks LED 3×3

### 3.1.1 Hasil Simulasi

Masukkan gambar rangkaian dan hasil simulasi pada bagian ini.




**Gambar 1:** Rangkaian simulasi matriks pushbutton dan LED




**Gambar 2:** Tampilan hasil simulasi matriks pushbutton dan LED

### 3.1.2 Source Code

Berikut merupakan program untuk membaca matriks pushbutton 3×3 dan mengendalikan LED yang bersesuaian. Setiap tombol mengubah kondisi LED pasangannya, sedangkan proses debouncing digunakan untuk mengurangi pembacaan akibat pantulan kontak.

```
#include <Arduino.h>

const uint8_t ROW_PINS[3] = {32, 33, 25};
const uint8_t COL_PINS[3] = {26, 27, 14};
const uint8_t LED_PINS[3][3] = {
    {23, 22, 21},
    {19, 18, 5},
    {17, 16, 4}
};

#define DEBOUNCE_DELAY 50

```

```
int lastReading[3][3];
int stableState[3][3];
unsigned long lastDebounceTime[3][3];
bool ledState[3][3];

void setup() {
    Serial.begin(115200);

    for (int r = 0; r < 3; r++) {
        pinMode(ROW_PINS[r], INPUT);
    }

    for (int c = 0; c < 3; c++) {
        pinMode(COL_PINS[c], INPUT_PULLUP);
    }

    for (int r = 0; r < 3; r++) {
        for (int c = 0; c < 3; c++) {
            pinMode(LED_PINS[r][c], OUTPUT);
            digitalWrite(LED_PINS[r][c], LOW);
            lastReading[r][c] = HIGH;
            stableState[r][c] = HIGH;
            lastDebounceTime[r][c] = 0;
            ledState[r][c] = false;
        }
    }

    Serial.println("Matrix 3x3 siap. Tekan tombol untuk toggle LED.");
}

void loop() {
    for (int r = 0; r < 3; r++) {
        pinMode(ROW_PINS[r], OUTPUT);
        digitalWrite(ROW_PINS[r], LOW);
        delayMicroseconds(10);

        for (int c = 0; c < 3; c++) {
            int reading = digitalRead(COL_PINS[c]);

            if (reading != lastReading[r][c]) {
                lastDebounceTime[r][c] = millis();
            }

            if ((millis() - lastDebounceTime[r][c]) > DEBOUNCE_DELAY) {
                if (reading != stableState[r][c]) {
                    stableState[r][c] = reading;

                    if (stableState[r][c] == LOW) {
                        ledState[r][c] = !ledState[r][c];
                        digitalWrite(LED_PINS[r][c],
                                     ledState[r][c] ? HIGH : LOW);

                        Serial.printf(
                            "Tombol (%d,%d) -> LED (%d,%d) %s\n",
```

```
                            r + 1, c + 1, r + 1, c + 1,
                            ledState[r][c] ? "ON" : "OFF"
                        );
                    }
                }
                lastReading[r][c] = reading;
            }
        }

        pinMode(ROW_PINS[r], INPUT);
    }
}
```

### 3.2 Pertanyaan Analisis

1. **Mengapa proses debouncing masih dibutuhkan pada matriks pushbutton, meskipun umumnya hanya satu tombol yang ditekan dalam satu waktu? Hubungkan penjelasan dengan penyebab terjadinya bouncing.**

   **Jawab:** Debouncing tetap diperlukan karena pantulan sinyal terjadi pada mekanisme kontak di dalam setiap tombol, bukan karena jumlah tombol yang ditekan. Ketika tombol ditekan atau dilepaskan, kontak logam dapat mengalami getaran singkat sehingga menghasilkan beberapa perubahan logika HIGH dan LOW dalam rentang beberapa milidetik. Proses pemindaian matriks yang berlangsung berulang dapat membaca setiap perubahan tersebut sebagai kejadian penekanan yang berbeda. Hal ini berpengaruh pada LED yang menggunakan mekanisme toggle: pantulan dengan jumlah genap dapat membuat LED kembali ke keadaan awal, sedangkan jumlah ganjil dapat menghasilkan kondisi akhir yang tidak konsisten. Penerapan debounce selama 50 ms membuat perubahan status baru diterima setelah sinyal berada pada kondisi stabil dalam waktu tersebut. Dengan demikian, satu penekanan fisik dapat diproses sebagai satu perintah.

2. **Jelaskan konfigurasi pull-up atau pull-down yang diterapkan pada kolom tombol, kondisi logika ketika tombol tidak ditekan, serta alasan pin tidak boleh berada dalam keadaan floating.**

   **Jawab:** Pada rancangan ini, ketiga pin kolom menggunakan pull-up internal melalui konfigurasi `INPUT_PULLUP`. Ketika tidak ada tombol yang ditekan, pin kolom ditarik menuju tegangan 3,3 V sehingga terbaca HIGH. Apabila tombol pada baris yang sedang diaktifkan ditekan, jalur kolom terhubung ke baris tersebut dan menghasilkan pembacaan LOW. Sementara itu, baris yang tidak sedang dipindai diatur sebagai INPUT agar berada dalam kondisi impedansi tinggi. Pengaturan tersebut membantu mencegah hubungan singkat ant baris ketika beberapa tombol pada kolom yang sama ditekan bersamaan. Kondisi floating perlu dihindari karena tegangan pin tidak memiliki level logika yang pasti dan dapat berubah akibat gangguan lingkungan atau noise. Pembacaan yang tidak menentu dapat menyebabkan sistem mendeteksi tombol seolah-olah sedang ditekan, meskipun tidak ada input dari pengguna.

3. **Jika sebuah tombol pada matriks diganti dengan LDR yang terhubung ke ADC, apa perbedaan pembacaan GPIO digital dan analog? Mengapa ADC mampu menunjukkan lebih dari dua keadaan?**

**Jawab:** GPIO yang dikonfigurasi sebagai digital input hanya mengenali dua keadaan logika, yaitu HIGH dan LOW. Mekanisme ini sesuai untuk pushbutton karena tombol pada dasarnya memiliki kondisi terhubung atau terputus. LDR memiliki karakteristik berbeda karena nilai resistansinya berubah mengikuti intensitas cahaya yang diterima. Perubahan resistansi tersebut menyebabkan tegangan pada titik pembagi tegangan ikut berubah secara bertahap. ADC 12-bit pada ESP32 mengubah tegangan analog menjadi nilai digital dengan 4096 kemungkinan tingkat, yaitu 0 sampai 4095. Oleh sebab itu, perubahan intensitas cahaya dapat direpresentasikan dalam banyak nilai, bukan hanya dua kondisi. LDR juga memerlukan pin ADC tersendiri, misalnya GPIO34, sehingga tidak dapat dipindai dengan cara yang sama seperti tombol pada matriks. Jika hasil pembacaan LDR ingin digunakan sebagai pemicu seperti tombol, program harus menentukan batas nilai ADC tertentu, misalnya LED dinyalakan ketika hasil pembacaan berada di bawah 1000.

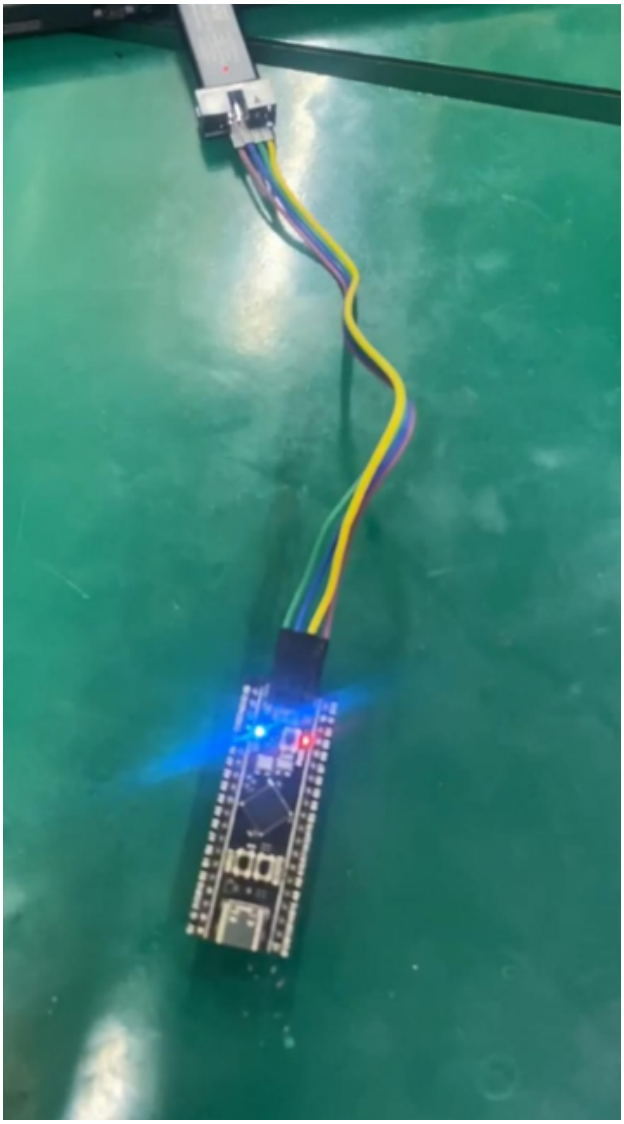
# 4 Kesimpulan

Berdasarkan pelaksanaan praktikum Modul 1, PlatformIO pada Visual Studio Code dapat digunakan untuk mengembangkan program pada STM32 Blackpill dan ESP32 dengan framework yang berbeda. Program Blink berhasil dijalankan pada STM32 melalui ST-Link menggunakan framework Arduino, serta pada ESP32 dengan framework ESP-IDF. Perbandingan keduanya memperlihatkan bahwa framework Arduino memiliki struktur program yang lebih sederhana, sedangkan ESP-IDF memberikan pengaturan GPIO dan FreeRTOS secara lebih langsung.

Pengujian pushbutton menunjukkan bahwa pull-up menghasilkan kondisi HIGH saat tombol dilepas dan LOW ketika tombol ditekan, sementara pull-down memberikan hasil sebaliknya. Kedua metode dapat diterapkan menggunakan resistor eksternal maupun fasilitas resistor internal ESP32. Selain itu, komunikasi serial memerlukan kesesuaian baud rate antara program dan Serial Monitor agar data dapat diterima dengan benar. Penerapan debouncing juga terbukti membantu mengurangi pembacaan tombol yang keliru akibat bouncing. Dari nilai yang diuji, waktu 50 ms memberikan hasil yang lebih seimbang dibandingkan 10 ms dan 200 ms. Pada pengujian LDR, ADC 12-bit mampu mengubah perubahan tegangan menjadi nilai digital dalam rentang 0–4095, sehingga dapat menggambarkan variasi intensitas cahaya dengan lebih rinci daripada masukan digital. Dengan demikian, praktikum ini memberikan pemahaman awal mengenai penggunaan GPIO sebagai digital output, digital input, dan analog input.

# 5 Lampiran

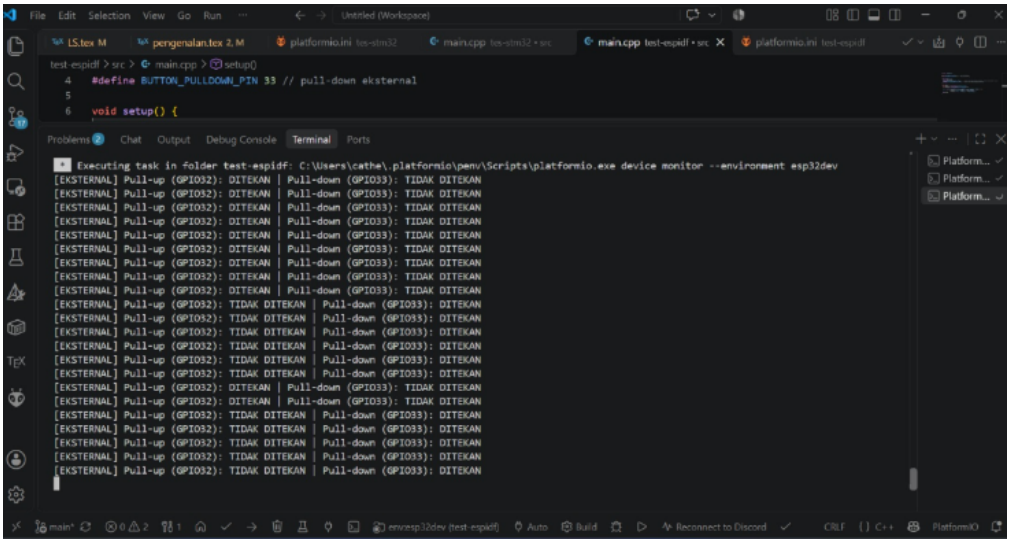
## 5.1 Dokumentasi Pelaksanaan Praktikum



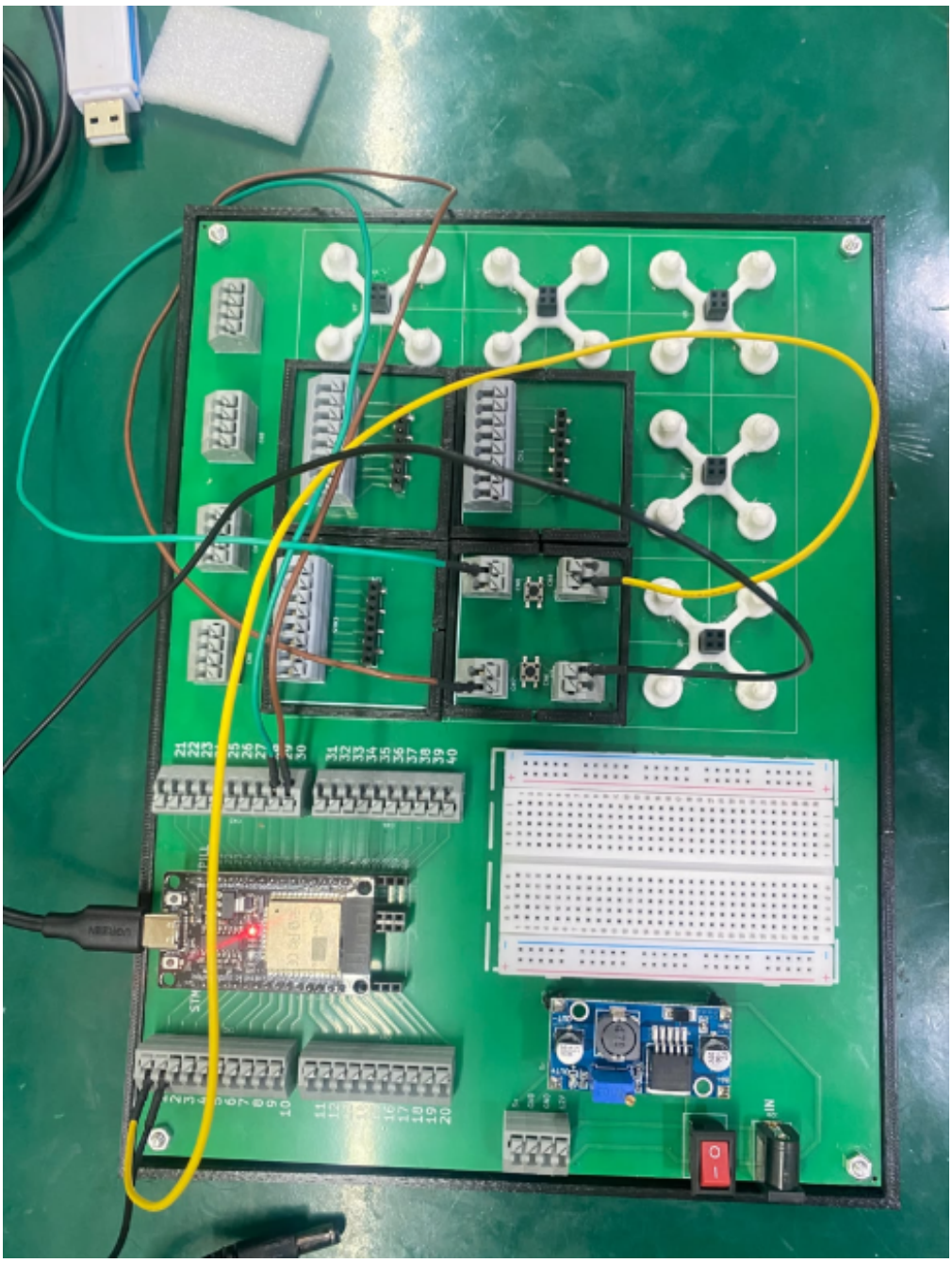
**Gambar 3:** Dokumentasi hasil pengujian Blink pada STM32 Blackpill menggunakan framework Arduino



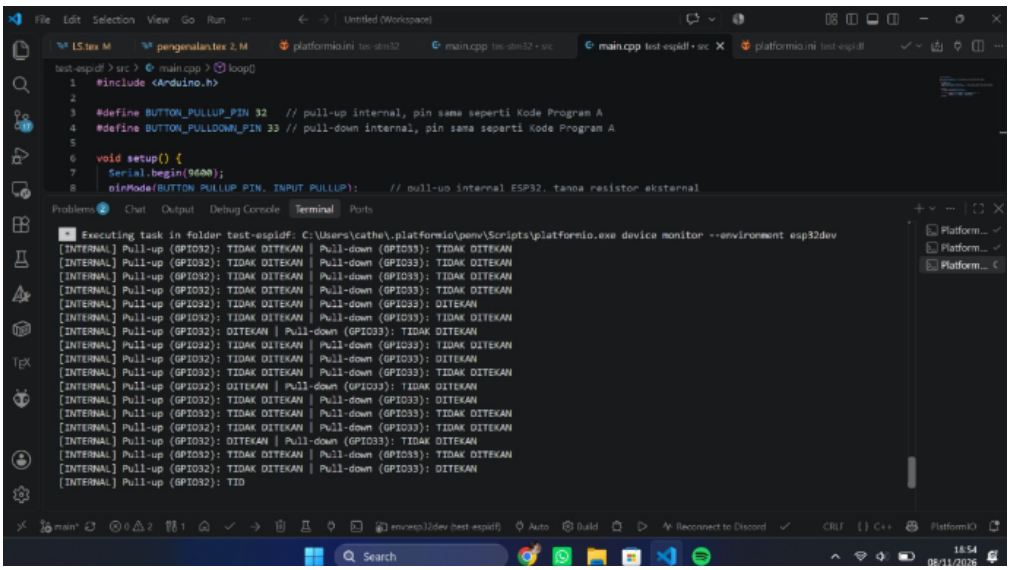
**Gambar 4:** Dokumentasi pengujian program Blink pada ESP32 dengan framework ESP-IDF



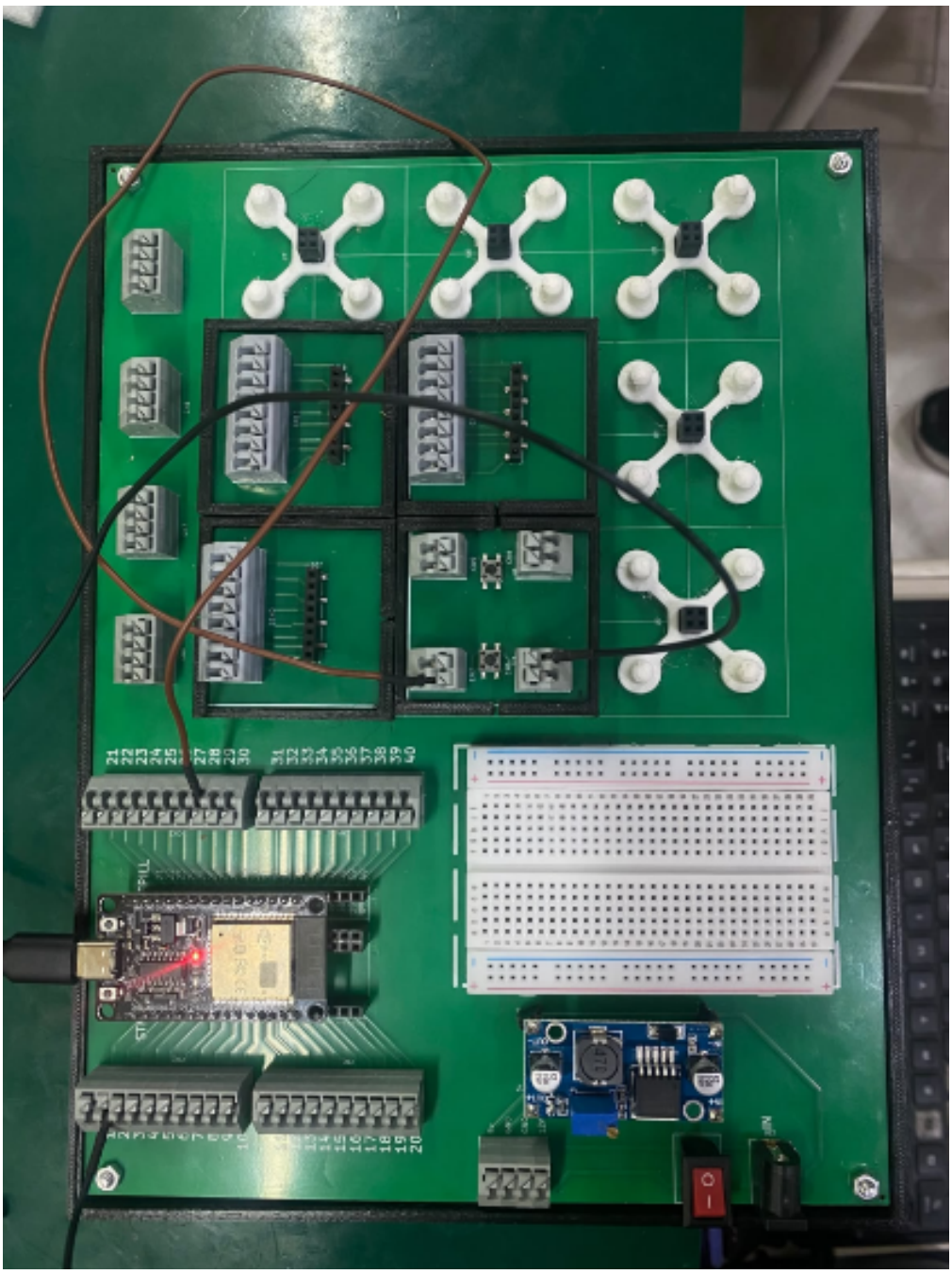

**Gambar 5:** Hasil pengamatan konfigurasi pull-up dan pull-down eksternal



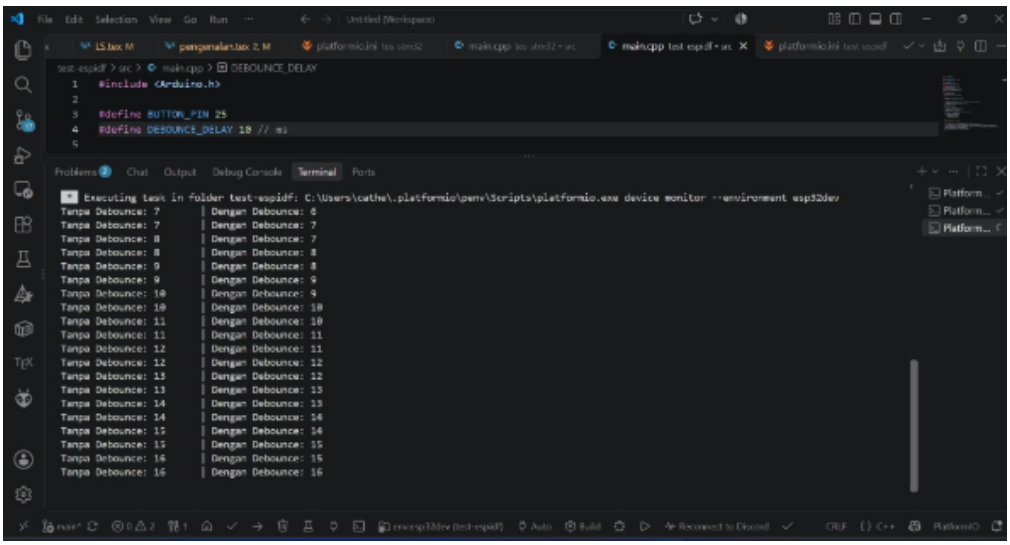
**Gambar 6:** Rangkaian pengujian pull-up dan pull-down internal



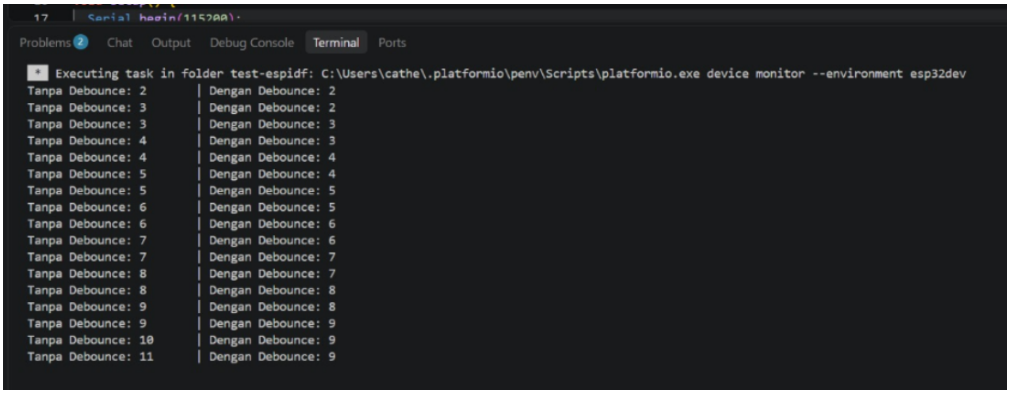

**Gambar 7:** Keluaran Serial Monitor pada pengujian pull-up dan pull-down internal



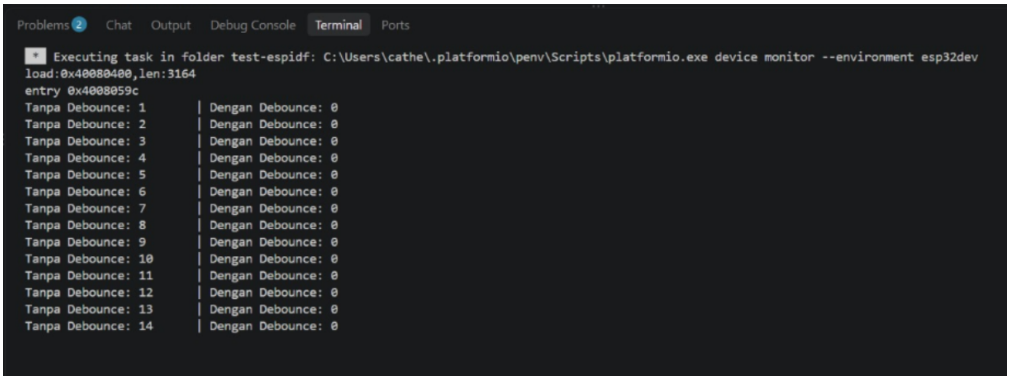
**Gambar 8:** Rangkaian pengujian debouncing pada input tombol GPIO



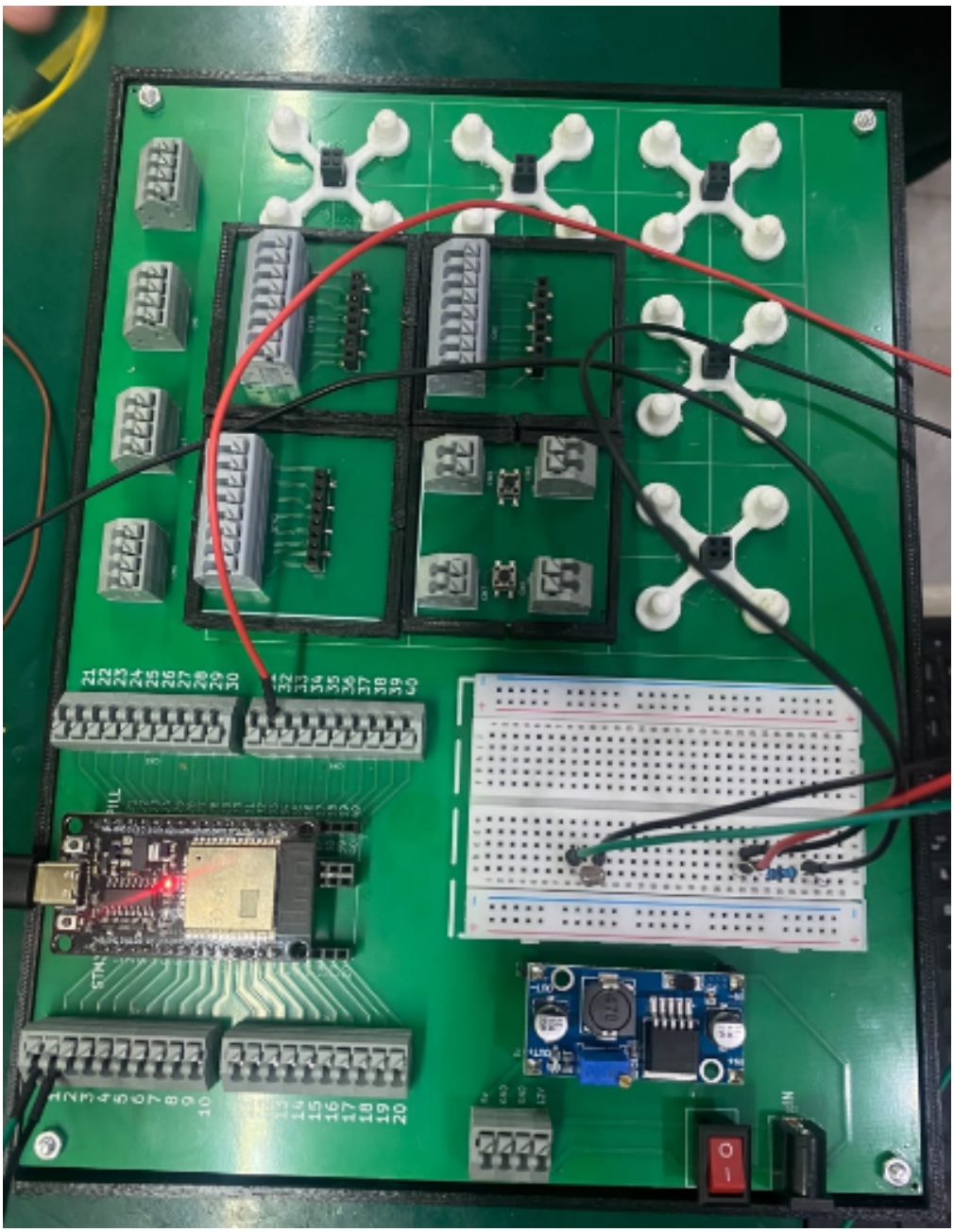

**Gambar 9:** Hasil pengujian debouncing dengan waktu tunda 10 ms



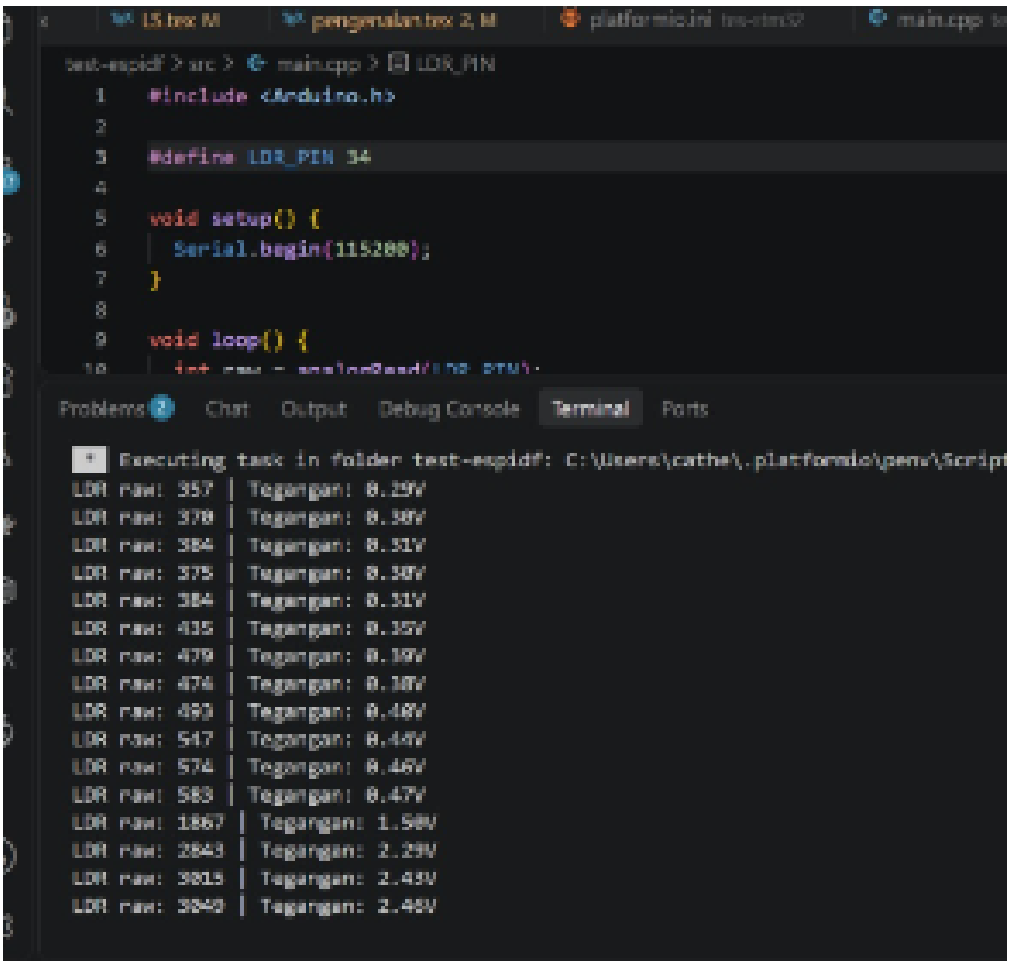

**Gambar 10:** Hasil pengujian debouncing dengan waktu tunda 50 ms




**Gambar 11:** Hasil pengujian debouncing dengan waktu tunda 200 ms



**Gambar 12:** Rangkaian percobaan pembacaan ADC menggunakan LDR




**Gambar 13:** Hasil pembacaan ADC pada LDR melalui Serial Monitor